DEPLOYMENT OF IRIS CLASSIFICATION MODEL USING FASTAPI, DOCKER AND AZURE REPORT

Introduction

In this lab, I worked on deploying a machine learning model using FastAPI, Docker, and Microsoft Azure. The main objective was to understand how a model developed locally can be converted into a real-world application that is accessible through a public URL.

The lab covered the complete workflow including model training, API creation, containerization, cloud deployment, and debugging.

Objectives

The objectives of this lab were:

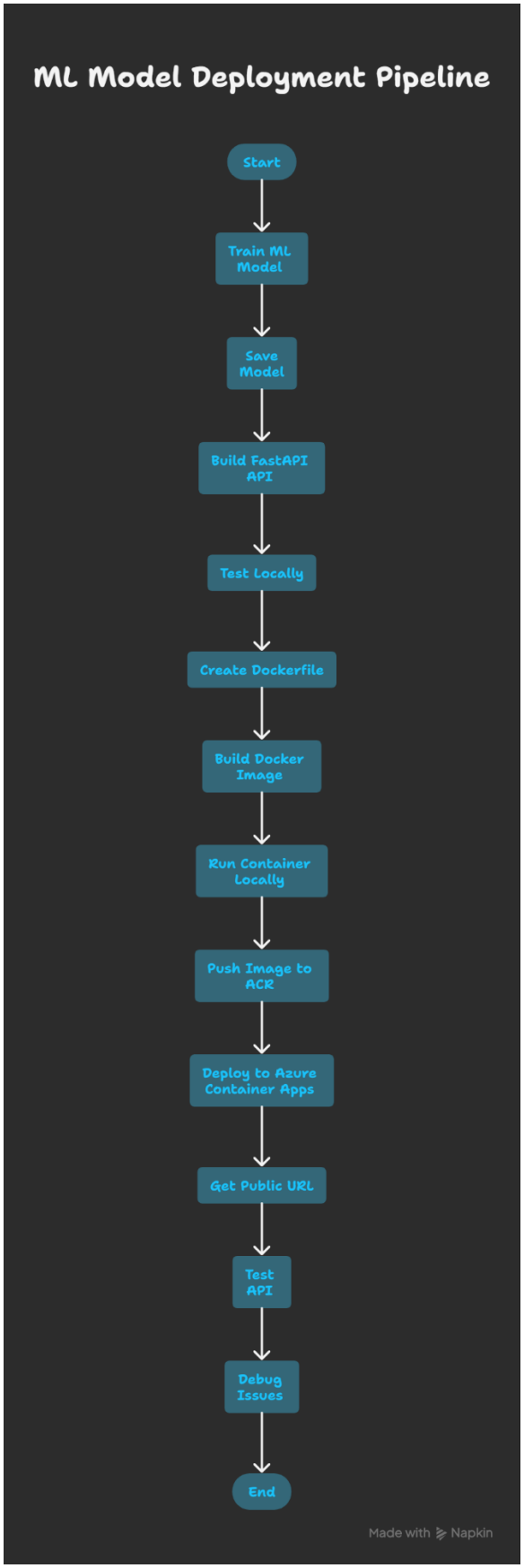
- To train a machine learning model using the Iris dataset
- To build an API using FastAPI
- To containerize the application using Docker
- To push the image to Azure Container Registry
- To deploy the application using Azure Container Apps
- To debug common deployment issues

Workflow

The overall workflow followed in this lab was:

Model Training → FastAPI API → Docker Image → Azure Container Registry → Azure Container Apps → Public API Endpoint

Figure: End-to-end workflow of model deployment using FastAPI, Docker, and Azure



Implementation and Issues Faced

Setup

I installed all required tools such as Python, Docker Desktop, and Azure CLI. I also logged into Azure and created a resource group.

Issues faced:

- Docker Desktop was not opening initially
- WSL setup showed no distributions

Resolution:

- Enabled virtualization and restarted the system
- Installed and configured WSL properly
- Ensured Docker Desktop was running before executing commands

Model Training

I trained a RandomForest model using the Iris dataset and saved it as iris_model.pkl.

Issues faced:

- Required libraries were missing

Resolution:

Installed dependencies using pip before running the training script.

Code Snippet :

```
from sklearn.ensemble import RandomForestClassifier

from sklearn.datasets import load_iris

import joblib


# Load dataset

iris = load_iris()
```

X, y = iris.data, iris.target

Train model

model = RandomForestClassifier()

model.fit(X, y)

Save model

joblib.dump(model, "iris_model.pkl")

Explanation:
A RandomForestClassifier was used because it is robust and performs well for classification tasks. The trained model was saved using joblib so it can be loaded later by the FastAPI application for predictions.

FastAPI Application

I created a FastAPI application with /predict and /health endpoints and tested it locally.

Issues faced:

- Model file could not be found initially
- Confusion with correct server port

Resolution:

- Fixed file path using proper Python path handling
- Ran the server correctly using uvicorn and tested using Swagger UI

Code Snippet (FastAPI API):

```
from fastapi import FastAPI

import joblib

app = FastAPI()


# Load trained model

model = joblib.load("iris_model.pkl")
```

```
@app.get("/health")
def health():
    return {"status": "ok"}
@app.post("/predict")
def predict(data: dict):
    features = [[
        data["sepal_length"],
        data["sepal_width"],
        data["petal_length"],
        data["petal_width"]
    ]]
    prediction = model.predict(features)
    return {"prediction": int(prediction[0])}
```

Explanation:
FastAPI is used to create REST API endpoints. The /predict endpoint accepts input data in JSON format and returns the predicted class. The /health endpoint is used to check if the API is running successfully.

Sample API Request and Response:

```
POST /predict
Input:
{
  "sepal_length": 5.1,
  "sepal_width": 3.5,
  "petal_length": 1.4,
  "petal_width": 0.2
}
Output:
{
  "prediction": 0
}
```

Docker

I created a Dockerfile, built the image, and tested it locally.

Issues faced:

- Docker daemon error occurred

Resolution:

- Started Docker Desktop before running Docker commands
- Verified container using local browser before moving to cloud

Code Snippet (Dockerfile):

```
FROM python:3.9
# Setting working directory inside container
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
# Command to run FastAPI app using Uvicorn
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8080"]
```

Azure Container Registry

I created an Azure Container Registry and pushed the Docker image.

Issues faced:

- Authentication error while logging into ACR
- Registry name conflict

Resolution:

- Logged in using Azure CLI before ACR login
- Used a unique registry name

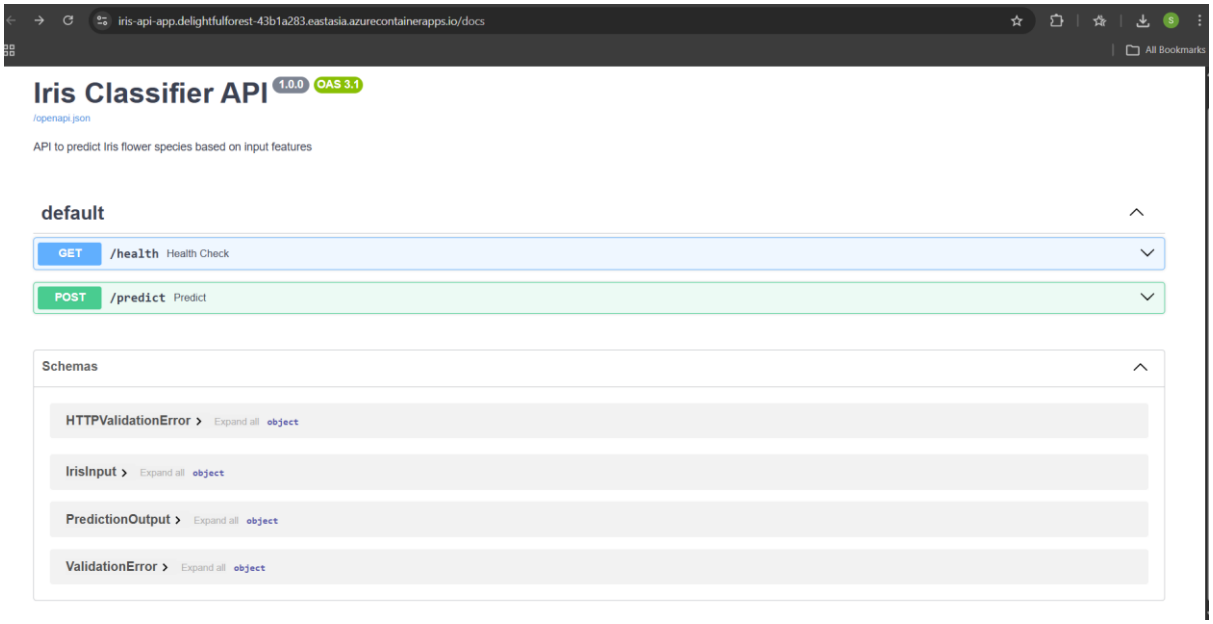
Deployment (Azure Container Apps)

I deployed the containerized application and exposed it using external ingress.

Public URL of deployed API:
https://iris-api-app.delightfulforest-43b1a283.eastasia.azurecontainerapps.io

The API was tested successfully using /docs and /predict.

Figure:Swagger UI



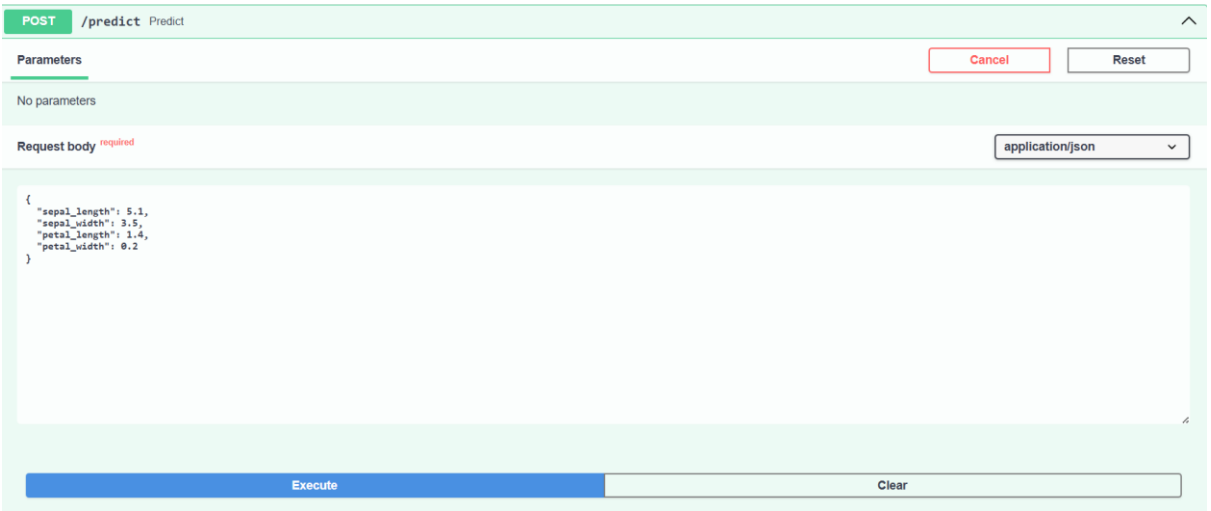
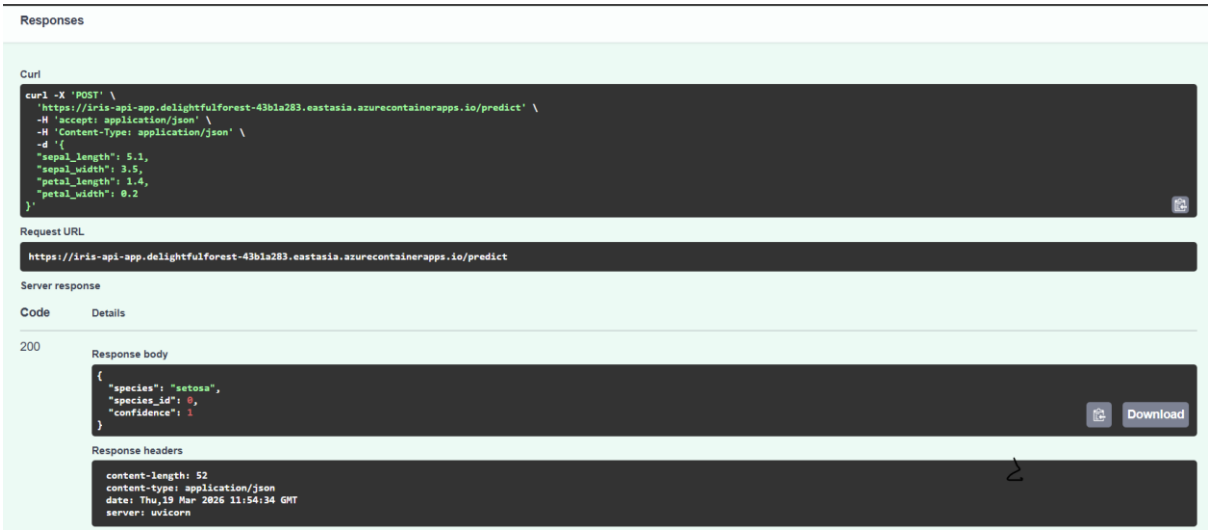


Figure: API prediction for Setosa input



Issues faced:

- Multi-line command error in Windows CMD
- Azure CLI extension issue
- Image pull authentication issue
- Confusion regarding correct port

Resolution:

- Used single-line commands instead of multi-line syntax
- Enabled dynamic extension installation
- Used ACR credentials for authentication
- Ensured application runs on port 8080

Debugging

This module focused on understanding how errors occur and how to debug them.

Issues faced:

- Changes in Dockerfile were not reflected after deployment
- Confusion between local changes and cloud deployment
- Unexpected behavior due to Docker caching

Resolution:

- Rebuilt the image without cache
- Followed correct deployment flow: build → push → update
- Used logs to analyze behavior

Observations:

- Port mismatch can cause application failure
- Missing files can cause runtime errors
- Invalid inputs are handled automatically by FastAPI
- Containerization ensures consistency between local and cloud environments
- APIs enable machine learning models to be accessed in real-world applications
- Proper port configuration is critical for cloud deployment

Revision History

I used the revision listing command to understand how Azure tracks deployments.

Understanding gained:

- Each deployment creates a new revision
- Previous versions are retained
- Applications can be rolled back to earlier versions

I understood that this concept is important for version control and safe deployment strategies.

Outcome of the Lab

- Successfully deployed a machine learning model to the cloud
- Created a working public API endpoint
- Verified predictions using different inputs
- Understood real-world deployment workflow
- Gained experience in debugging deployment issues
- Learned how cloud systems behave differently from local environments

Conclusion

This lab helped me understand the complete process of deploying a machine learning model from development to production. I learned how to use FastAPI, Docker, and Azure services together, and also how to debug issues that occur during deployment.

The hands-on experience gained in this lab improved my understanding of real-world MLOps workflows and cloud-based applications.

Note: AI tools (ChatGPT and Claude) were used to assist in structuring the report, improving clarity, and refining explanations.

Used Napkin AI for generating workflow flowchart image.